Menu
- Home
- Learn Applications Solutions
  - Bluetooth
  - Controller Area Network (CAN)
  - Embedded Security
  - Ethernet
  - I2C
  - Industrial Communications
  - Universal Serial Bus (USB)
  - Linux 4 SAM
    - Applications
    - Base Components
    - Boards
      - Sam9x60 Curiosity
      - Sam9x60EK
      - Sam9x75 Curiosity
      - SAM9X75 Curiosity Pro
      - Sama5D2 PTC EK
      - Sama5d2 Xplained
      - Sama5d2-ICP
      - Sama5d27Som1-EK
      - Sama5d27WLSom1-EK
      - Sama5d29 Curiosity
      - Sama5d3 Xplained
      - Sama5d4 Xplained
      - SAMA7D65-Curiosity
      - SAMA7D65 Curiosity Pro
      - Sama7g5-EK
      - Older Boards
        - AT91SAM9x5 EK
        - Sama5D3X EK
        - Sama5D4 EK
    - Build Systems
    - Demo Archive for Linux 4 Sam from Microchip
    - FAQ
    - WebFAQ
  - LoRa®
  - Local Interconnect Network (LIN)
  - Machine Learning
  - Motor Control
  - Power Management and Conversion
  - Signal Conditioning
  - What is Single Wire Interface (SWI) Protocol?
  - SPI Introduction
  - TCP/IP
  - Temperature Management
  - Touch Sensing Solutions
  - What is Single Wire Interface (SWI) Protocol?
  - Wi-Fi®
- Learn Products
- Learn Software and Hardware Tools
- Site Information

/ Learn Applications Solutions at Microchip / … / Boards / Older Boards / AT91SAM9x5 EK

AT91SAM9x5 EK

Last modified by Microchip on 2026/01/09 08:12

On This Page
- SoC Features
- Kit Information
  - Kit Overview
  - Access the console
  - Access the console on DBGU serial port
    - Using RS-232 connector (DBGU J11)
- Demo
  - Demo archives
  - Create a SD card with the demo
    - Multi-Platform Procedure
  - Flash the demo
  - Connect the USB to the board before launching SAM-BA
    - Run script to flash the demo
- Build From source code
  - Setup ARM Cross Compiler
  - Build AT91Bootstrap from sources
    - Get AT91Bootstrap Source Code
    - Configure AT91Bootstrap
    - Customize AT91Bootstrap
    - Build AT91Bootstrap
  - Build U-Boot from sources
    - Getting U-Boot sources
    - Cross-compiling U-Boot
  - Build Kernel from sources
    - Required packages
    - Getting Kernel sources
    - Setup ARM Cross Compiler
    - Configure and Build the Linux kernel
  - Build Yocto Project rootfs from sources
    - Building Environment
      - Step-by-Step Build Procedure
        - OpenEmbedded/Yocto Project BSP Layer for Microchip's SoCs
- Using SAM-BA to flash components to board
  - NAND Flash Demo - Memory Map
  - Install SAM-BA Software In Your PC
  - Launch SAM-BA tools
  - Configure NAND ECC
    - Using default PMECC parameters
  - Programming components into NAND
    - Program AT91Bootstrap binary
    - Program U-Boot binary
- Recent FAQ

Microchip Support

Query Microchip Forums and get your questions answered by our community:

Microchip Forums
AVR Freaks Forums

If you need to work with Microchip Support staff directly, you can submit a technical support case. Keep in mind that many questions can be answered through our self-help resources, so this may not be your speediest option.

Technical Support Portal

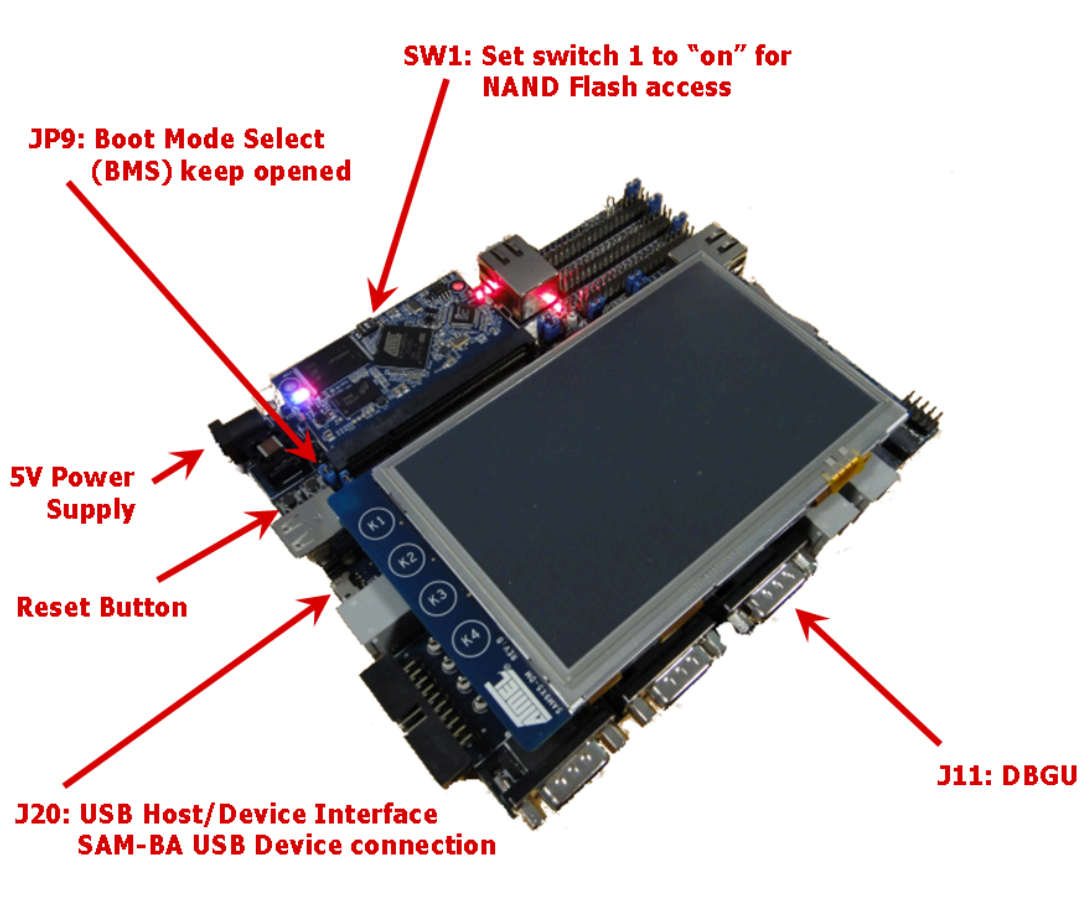
SoC Features

The SAM9x5 family is based on the ARM ARM926EJ-S core. The SAM9x5 family runs at 400 MHz and features connectivity peripherals, high data bandwidth architecture, and a small footprint package option, making it an optimized solution for industrial applications. Peripherals include a soft modem supporting exclusively the Conexant SmartDAA line driver, High Speed (480Mbps) USB Host/Device ports and Full Speed USB Host with on-chip transceivers, 10/100 Ethernet MAC, two HS SDCard/SDIO/MMC interfaces, USARTs, SPIs, I2S, multiple TWIs and 10-bit ADC. It offers support for 8-bank DDR2 and for LPDDR, SDRAM/LPSDRAM, static memories, as well as specific circuitry for MLC/SLC NAND Flash with integrated ECC up to 24 bits. The SAM9x5 family is available in a 217-ball BGA package with 0.8mm ball pitch, as well as a 247-ball BGA package with 0.5mm ball pitch.

SAM9G15
SAM9G25
SAM9G35
SAM9X25
SAM9X35
ARM926
Connectivity
System
User Interface
Control
Memory

Kit Information

Kit Overview


SW1: Set switch 1 to "on" for NAND Flash access
JP9: Boot Mode Select (BMS) keep opened
5V Power Supply
Reset Button
J20: USB Host/Device Interface SAM-BA USB Device connection
J11: DBGU

Access the console

The usual serial communication parameters are 115200 8-N-1 :

Baud rate	115200
Data	8 bits
Parity	None
Stop	1 bit
Flow control	None

Access the console on DBGU serial port

The DBGU serial console can be accessed from the RS-232 connector (marked as DBGU J11).

Using RS-232 connector (DBGU J11)

- Connect a DB9 serial cable to the J11 connector
- Now open your favorite terminal emulator with appropriate settings
  - **For Linux users:** identify the USB connection by monitoring the last lines of dmesg command. The /dev/ttyUSBx or /dev/ttyACMx number will be used to configure the terminal emulator.

```
[605576.562740] usb 1-1.1.2: new full-speed USB device number 17 using ehci-pci
[605576.660920] usb 1-1.1.2: New USB device found, idVendor=0403, idProduct=6001
[605576.660933] usb 1-1.1.2: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[605576.660939] usb 1-1.1.2: Product: TTL232R-3V3
[605576.660944] usb 1-1.1.2: Manufacturer: FTDI
[605576.660958] usb 1-1.1.2: SerialNumber: FTGNVZ04
[605576.663092] ftdi_sio 1-1.1.2:1.0: FTDI USB Serial Device converter detected
[605576.663120] usb 1-1.1.2: Detected FT232RL
[605576.663122] usb 1-1.1.2: Number of endpoints 2
[605576.663124] usb 1-1.1.2: Endpoint 1 MaxPacketSize 64
[605576.663126] usb 1-1.1.2: Endpoint 2 MaxPacketSize 64
[605576.663128] usb 1-1.1.2: Setting MaxPacketSize 64
[605576.663483] usb 1-1.1.2: FTDI USB Serial Device converter now attached to ttyUSB0
```

A /dev/ttyUSB0 node has been created.

- Now open your favorite terminal emulator with appropriate settings

Demo

Demo archives

Media type	Board	Screen	Binary	Description
Yocto Project / Poky based demo				
NAND Flash	AT91SAM9x5-EK	-	linux4sam-poky-at91sam9x5ek-6.1.zip (~ 102 MB) md5: e83868ae1f9ea1b77c31e2d9e58fd969	Linux4SAM Yocto Project / Poky based demo compiled from tag **linux4sam_6.1** Follow procedure: Flash the demo
SD Card image	AT91SAM9x5-EK	-	linux4sam-poky-at91sam9x5ek-6.1.img.bz2 (~ 80 MB) md5: d6c0d2da6c30086a7b6763c3d7bdc202	Linux4SAM Yocto Project / Poky based demo compiled from tag **linux4sam_6.1** Follow procedure: Create a SD card with the demo
BuildRoot based demo				
SD Card image	AT91SAM9x5-EK	-	linux4sam-buildroot-at91sam9x5ek-6.1.img.bz2 (~ 52 MB) md5: 6d1927f815ee6473b7b3c7b556888933	Linux4SAM BuildRoot based demo compiled from tag **linux4sam_6.1** Follow procedure: Create a SD card with the demo

Create a SD card with the demo

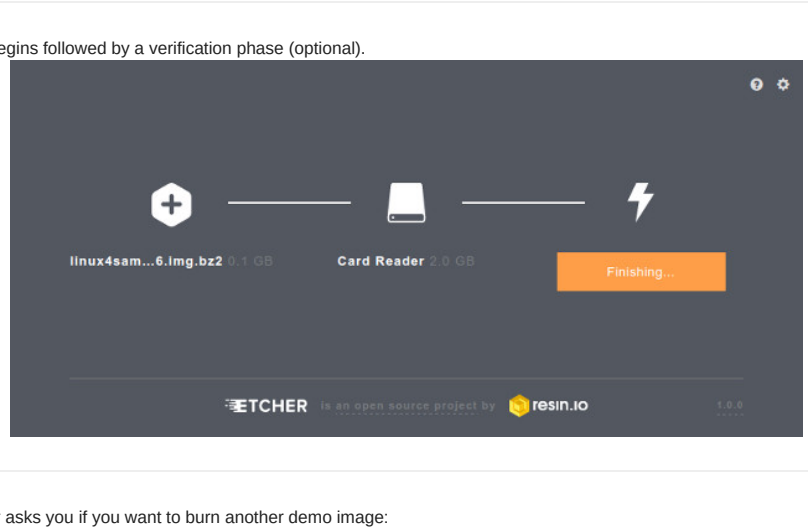
You need a 1 GB (or larger) SD card and to download the image of the demo. The image is compressed to reduce the amount of data to download. This image contains:

- a FAT32 partition with the AT91Bootstrap, U-Boot and the Linux Kernel (zImage and dtb), and
- an EXT4 partition for the rootfs.

Multi-Platform Procedure

To write the compressed image on the SD card, you will have to download and install balenaEtcher. This open-source software tool is useful for obtaining a compressed image as input. Additional information and support can be found on the balenaEtcher website.

1. Insert your SD card and launch **Etcher**:



2. Select the demo image.

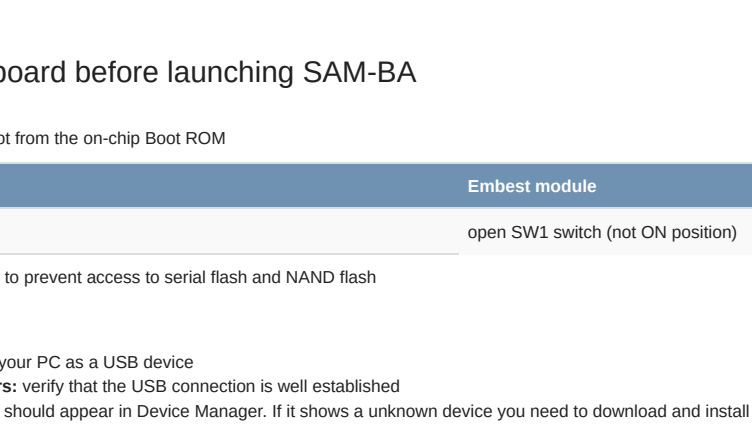
   Note: You can select a compressed image. The tool can decompress files "on the fly."

3. Select the device corresponding to your SD card (Etcher suggests the devices that are removable to avoid erasing your system disk).

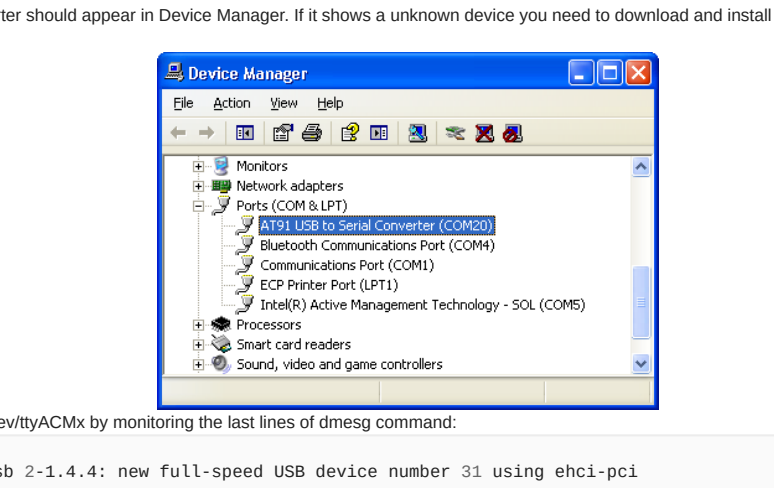
4. Click on the **Flash!** button.

5. On Linux, Etcher finally asks you to enter your root password because it needs access to the hardware (your SD card reader or USB to SD card converter).

6. Then the flashing process begins followed by a verification phase (optional).



7. Once writing is done, Etcher asks you if you want to burn another demo image:



   Your SD card is ready!

Flash the demo

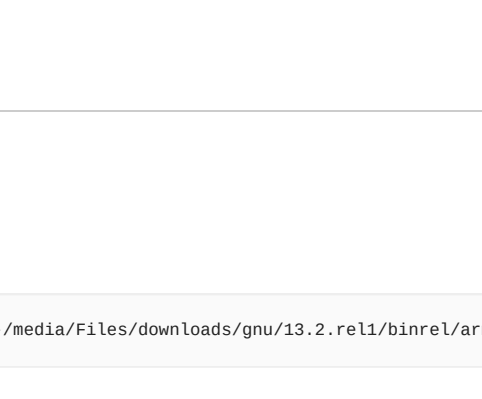
Use SAM-BA software v3.9.x or onwards. You can download SAM-BA software v3.91 from the release page.

Connect the USB to the board before launching SAM-BA

- Remove power from the board
- JP9 must be opened (BMS=1) to boot from the on-chip Boot ROM

Cogent module	Embest module	Ronetix module
open *NCS jumper	open SW1 switch (not ON position)	open J1 and J2 jumpers
which is (are) located on the CPU module to prevent access to serial flash and NAND flash		

- power up the board
- check whether the board is found in your PC as a USB device
  - **For Microsoft Windows users:** verify that the USB connection is well established
    AT91 USB to Serial Converter should appear in Device Manager. If it shows a unknown device you need to download and install the driver: AT91SAM USB CDC driver


Device Manager
File
Action
View
Help
Monitors
Network adapters
Ports (COM & LPT)
AT91 USB to Serial Converter (COM20)
Bluetooth Communications Port (COM4)
Communications Port (COM1)
ECP Printer Port (LPT1)
Intel(R) Active Management Technology - SOL (COM5)
Processors
Smart card readers
Sound, video and game controllers

  - **For Linux users:** check /dev/ttyACMx by monitoring the last lines of dmesg command:

```
[172677.700868] usb 2-1.4.4: new full-speed USB device number 31 using ehci-pci
[172677.792677] usb 2-1.4.4: not running at top speed; connect to a high speed hub
[172677.793418] usb 2-1.4.4: New USB device found, idVendor=03eb, idProduct=6124
[172677.793424] usb 2-1.4.4: New USB device strings: Mfr=0, Product=0, SerialNumber=0
[172677.793897] cdc_acm 2-1.4.4:1.0: This device cannot do calls on its own. It is not a modem.
[172677.793924] cdc_acm 2-1.4.4:1.0: ttyACM0: USB ACM device
```

idVendor=03eb, idProduct=6124: from this message you can see it's Microchip board USB connection.

Cogent module	Embest module	Ronetix module
close *NCS jumper	close SW1 switch (move to ON position)	close J1 and J2 jumpers
which is (are) located on the CPU module to re-gain access to serial flash and NAND flash		

Run script to flash the demo

- download the demo package for the board. They are marked as "Media type: NAND Flash " in the table above
- extract the demo package
- run your usual terminal emulator and enter the demo directory
- make sure that the sam-ba application is in your Operating System path so that you can reach it from your demo package directory
- for Microsoft Windows users: Launch the demo_linux_nandflash.bat file
- for Linux users: Launch the demo_linux_nandflash.sh file
- this script runs SAM-BA 3 and the associated QML sam-ba script (demo_linux_nandflash_usb.qml) with proper parameters
- when you reach the end of the flashing process (this will take a few minutes), the following line is written:
  `-I- === Done. ===`
- connect a serial link on DBGU and open the terminal emulator program as explained just above
- ower cycle the board
- monitor the system while it's booting on the LCD screen or through the serial line

Build From source code

Setup ARM Cross Compiler

1. Download the **Arm GNU toolchain**:

```
wget -c https://developer.arm.com/-/media/Files/downloads/gnu/13.2.rel1/binrel/arm-gnu-toolchain-13.2.rel1-x86_64-arm
```

Back to Top

2. Add the **Arm GNU toolchain** to your system:

```
tar -xf arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-linux-gnueabihf.tar.xz
export CROSS_COMPILE=`pwd`/arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-
```

or

```
tar -xf arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-linux-gnueabihf.tar.xz
export CROSS_COMPILE=arm-none-linux-gnueabihf-
export PATH=$PATH:/YOUR/PATH/TO/arm-gnu-toolchain-13.2.Rel1-x86_64-arm-none-linux-gnueabihf/bin/
```

If you already have an old Arm GNU toolchain, you need to clean up the PATH with:

```
export PATH=${PATH/':/YOUR/PATH/TO/arm-gnu-toolchain-VERSION-x86_64-arm-none-linux-gnueabihf/bin/'/}
```

Back to Top

Build AT91Bootstrap from sources

This section describes how to get source code from the git repository, how to configure with the default configuration, how to customize AT91Bootstrap based on the default configuration and finally to build AT91Bootstrap to produce the binary. take the default configuration to download U-Boot from NandFlash for example.

Get AT91Bootstrap Source Code

You can easily download AT91Bootstrap source code on the at91bootstrap git repository.

To get the source code, you should clone the repository by doing:

```
$ git clone https://github.com/linux4sam/at91bootstrap.git
Cloning into 'at91bootstrap'...
remote: Enumerating objects: 17621, done.
remote: Counting objects: 100% (3324/3324), done.
remote: Compressing objects: 100% (1029/1029), done.
remote: Total 17621 (delta 2465), reused 3102 (delta 2285), pack-reused 14297
Receiving objects: 100% (17621/17621), 5.65 MiB | 4.65 MiB/s, done.
Resolving deltas: 100% (13459/13459), done.
$ cd at91bootstrap/
```

Configure AT91Bootstrap

Assuming you are at the AT91Bootstrap root directory, you will find a board/at91sam9x5ek folder which contains several default configuration files:

```
at91sam9x5eksd_uboot_defconfig
at91sam9x5ekdf_uboot_defconfig
at91sam9x5eknf_uboot_defconfig
```

nf means to read nandflash, df means to read serial flash, sd means to read mmc card.

linux means to load linux kernel to RAM, android means to load android kernel to RAM, uboot means to load u-boot to RAM, dt means to load dtb to RAM.

You can configure AT91Bootstrap to load U-Boot binary from NAND flash by doing:

```
$ make mrproper
$ make at91sam9x5eknf_uboot_defconfig
```

If the configuring process is successful, the .config file can be found at AT91Bootstrap root directory.

Customize AT91Bootstrap

If the default configuration doesn't meet your need, after configuring with the default configuration, you can customize it by doing:

```
$ make menuconfig
```

Now, in the menuconfig dialog, you can easily add or remove some features to/from AT91Bootstrap as the same way as kernel configuration.
Move to <Exit> with arrows and press this button hitting the Enter key to exit from this screen.

Build AT91Bootstrap

Then you can build the AT91Bootstrap binary by doing:

```
$ make
```

If the building process is successful, the final .bin image is *build/binaries/at91bootstrap.bin*.

Build U-Boot from sources

Getting U-Boot sources

Dedicated page on U-Boot wiki: http://www.denx.de/wiki/U-Boot/SourceCode

You can easily download U-Boot source code from Linux4Microchip GitHub U-Boot repository:

- clone the Linux4microchip GitHub U-Boot repository

```
    $ git clone https://github.com/linux4microchip/u-boot-mchp.git
Cloning into 'u-boot-mchp'...
remote: Enumerating objects: 951876, done.
remote: Counting objects: 100% (17718/17718), done.
remote: Compressing objects: 100% (5735/5735), done.
remote: Total 951876 (delta 12391), reused 15314 (delta 11846), pack-reused 934158
Receiving objects: 100% (951876/951876), 164.77 MiB | 401.00 KiB/s, done.
Resolving deltas: 100% (790362/790362), done.
  $ cd u-boot-mchp/
```

- The source code has been taken from the *master* branch which is pointing to the latest branch we use. If you want to use the other branch, you can list them and use one of them by doing:

```
  $ git branch -r
 origin/HEAD -> origin/master
 origin/dev/tony/sama7g5ek_optee
 origin/master
 origin/sam9x60_curiosity_early
 origin/sam9x60_early
 origin/sam9x60_iar
 origin/sam9x7_early
 origin/sama5d27wlsom1ek_ear
 origin/sama7g5_early
 origin/u-boot-2012.10-at91
 origin/u-boot-2013.07-at91
 origin/u-boot-2014.07-at91
 origin/u-boot-2015.01-at91
 origin/u-boot-2016.01-at91
 origin/u-boot-2016.03-at91
 origin/u-boot-2017.03-at91
 origin/u-boot-2018.07-at91
 origin/u-boot-2019.04-at91
 origin/u-boot-2020.01-at91
 origin/u-boot-2021.04-at91
 origin/u-boot-2022.01-at91
 origin/u-boot-2023.07-mchp
 origin/u-boot-2024.07-mchp
 origin/uboot_5series_1.x

  $ git checkout origin/u-boot-2024.07-mchp -b u-boot-2024.07-mchp
 Branch 'u-boot-2024.07-mchp' set up to track remote branch 'u-boot-2024.07-mchp' from 'origin'.
 Switched to a new branch 'u-boot-2024.07-mchp'
```

Cross-compiling U-Boot

Before compiling the U-Boot, you need setup cross compile toolchain in the section.

Latest versions of U-boot (2018.07 and newer) have a minimum requirement of 6.0 version of the GCC toolchain. We always recommend to use the latest versions.

Once the AT91 U-Boot sources available, cross-compile U-Boot is made in two steps: configuration and compiling. Check the Configuration chapter in U-Boot reference manual.

Go to the configs/ to find the exact target when invoking *make*.

The U-Boot environment variables can be stored in different media, above config files can specify where to store the U-Boot environment.

```
  # To put environment variables in serial flash:
  at91sam9x5ek_spiflash_defconfig
  # To put environment variables in nandflash (default):
  at91sam9x5ek_nandflash_defconfig
  # To put environment variables in SD/MMC card:
  at91sam9x5ek_mmc_defconfig
```

Here are the building steps for the AT91SAM9x5-EK board:

```
 # You can change the config according to your needs.
 make at91sam9x5ek_nandflash_defconfig
 make
```

The result of these operations is a fresh U-Boot binary called u-boot.bin corresponding to the binary ELF file u-boot.

- u-boot.bin is the file you should store on the board
- u-boot is the ELF format binary file you may use to debug U-Boot through a JTag link for instance.

Build Kernel from sources

Required packages

You must install essential host packages on your build host. These requirements are listed in the Linux kernel documentation with the chapter **Install build requirements**. You must follow this process which includes, but not limited to, the following packages:

- build-essential
- flex
- bison
- git
- perl-base
- libssl-dev
- libncurses5-dev
- libncursesw5-dev
- ncurses-dev

Getting Kernel sources

To get the source code, you have to clone the repository:

```
$ git clone https://github.com/linux4microchip/linux.git
Cloning into 'linux'...
remote: Enumerating objects: 8587836, done.
remote: Total 8587836 (delta 0), reused 0 (delta 0), pack-reused 8587836
Receiving objects: 100% (8587836/8587836), 3.49 GiB | 13.44 MiB/s, done.
Resolving deltas: 100% (7117887/7117887), done.
Updating files: 100% (70687/70687), done.
$ cd linux
```

The source code has been taken from the master branch which is pointing on the latest branch we use.

Note that you can also add this Linux4SAM repository as a remote GIT repository to your usual Linux git tree. It will save you a lot of bandwidth and download time:

```
$ git remote add linux4microchip https://github.com/linux4microchip/linux.git
$ git remote update linux4microchip
Fetching linux4microchip
From https://github.com/linux4microchip/linux
 * [new branch]              linux-6.6-mchp -> linux4microchip/linux-6.6-mchp
 * [new branch]              linux-6.12-mchp -> linux4microchip/linux-6.12-mchp
 * [new branch]              master     -> linux4microchip/master
```

If you want to use another branch, you can list them and use one of them by doing this:

```
$ git branch -r
  linux4microchip/linux-6.1-mchp
  linux4microchip/linux-6.1-mchp+fpga
  linux4microchip/linux-6.6-mchp
  linux4microchip/linux-6.6-mchp+fpga
  linux4microchip/linux-6.12-mchp
  linux4microchip/master
$ git checkout -b linux-6.12-mchp --track remotes/linux4microchip/linux-6.12-mchp
Branch linux-6.12-mchp set up to track remote branch linux-6.12-mchp from linux4microchip.
Switched to a new branch 'linux-6.12-mchp'
```

Setup ARM Cross Compiler

- First step is to dowload the ARM GNU Toolchain:

```
wget -c https://developer.arm.com/-/media/Files/downloads/gnu/13.2.rel1/binrel/arm-gnu-toolchain-13.2.rel1-x86_64-arm-non
```

- Next step is to add the ARM GNU Toolchain into your system:

```
tar -xf arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-linux-gnueabihf.tar.xz
export CROSS_COMPILE=`pwd`/arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-linux-gnueabihf/bin/arm-none-linux-gnueabihf-
```

or

```
tar -xf arm-gnu-toolchain-13.2.rel1-x86_64-arm-none-linux-gnueabihf.tar.xz
export CROSS_COMPILE=arm-none-linux-gnueabihf-
export PATH=$PATH:/YOUR/PATH/TO/arm-gnu-toolchain-13.2.Rel1-x86_64-arm-none-linux-gnueabihf/bin/
```

  - If you already have an old ARM GNU Toolchain need to clean up the PATH with:

```
export PATH=${PATH/':/YOUR/PATH/TO/arm-gnu-toolchain-VERSION-x86_64-arm-none-linux-gnueabihf/bin/'/}
```

Configure and Build the Linux kernel

Now you have to configure the Linux kernel according to your hardware. We have two default configuration at91 SoC in arch/arm/configs

```
arch/arm/configs/at91_dt_defconfig
arch/arm/configs/sama5_defconfig
arch/arm/configs/sama7_defconfig
```

- at91_dt_defconfig: for SAM9 (ARM926) series chips
- sama5_defconfig: for SAMA5 series chips
- sama7_defconfig: for SAMA7 series chips

You can modify default configuration using the menuconfig.

```
$ make ARCH=arm menuconfig
```

You can add or remove some features in the menuconfig dialog. Once done, use the arrows to navigate to **<Exit>** and press this button with the **Enter** key to exit from this screen.

Build the Linux kernel image, but before you build, you need to set up the cross compile toolchain. Check the "Set Up Arm Cross Compiler" section.

```
$ make ARCH=arm

[..]

  Kernel: arch/arm/boot/Image is ready
  Kernel: arch/arm/boot/zImage is ready
```

Now you have a usable compressed kernel image zImage.

If you need a uImage, you can run this additional step:

```
make ARCH=arm uImage LOADADDR=0x20008000

[..]

Kernel: arch/arm/boot/Image is ready
Kernel: arch/arm/boot/zImage is ready
UIMAGE  arch/arm/boot/uImage
Image Name:   Linux-6.12.22-linux4microchip-20
Created:      Thu May 22 18:05:21 2025
Image Type:   ARM Linux Kernel Image (uncompressed)
Data Size:    5688984 Bytes = 5555.65 KiB = 5.43 MiB
Load Address: 20008000
Entry Point:  20008000
Kernel: arch/arm/boot/uImage is ready
```

```
make ARCH=arm dtbs

[..]

  DTC     arch/arm/boot/dts/microchip/at91-sama5d27_som1_ek.dtb
  DTC     arch/arm/boot/dts/microchip/at91-sama5d27_wlsom1_ek.dtb
  DTC     arch/arm/boot/dts/microchip/at91-sama5d29_curiosity.dtb
  DTC     arch/arm/boot/dts/microchip/at91-sama5d2_icp.dtb
  DTC     arch/arm/boot/dts/microchip/at91-sama5d3_eds.dtb
  DTC     arch/arm/boot/dts/microchip/at91-sama7d65_curiosity.dtb
  DTC     arch/arm/boot/dts/microchip/at91-sama7g54_curiosity.dtb
  DTC     arch/arm/boot/dts/microchip/at91-sama7g5ek.dtb

[..]
```

If the building process is successful, the final images can be found under the arch/arm/boot/ directory.

Build Yocto Project rootfs from sources

Note: Building an entire distribution is a long process. It also requires a large amount of free disk space.

Support for the Microchip MPU System-on-a-Chip (SoC) family is provided through the meta-mchp Yocto Project® layer. The source for this layer is hosted on the Linux4Microchip GitHub® account on the Microchip Yocto Project BSP page.

Building Environment

A step-by-step, comprehensive installation is explained on the Yocto Project Quick Build page. The following lines should be considered as an add-on that is MPU-specific, or that can facilitate your setup.

Step-by-Step Build Procedure

Note: The README procedure is available directly in the meta-mchp-common layer. This file in the meta-mchp layer repository should be considered the reference, and the following copy may be out of sync.

Note: Starting with the Linux4Microchip 2025.04 release, the meta-mchp layer supports Yocto Project templates, so make sure you create a new build environment using oe-init-build-env.

OpenEmbedded/Yocto Project BSP Layer for Microchip's SoCs

- Description
  - The meta-mchp-common layer consolidates common Board Support Package (BSP) components and metadata for Microchip platforms, streamlining development across various Microchip devices for use with OpenEmbedded and/or Yocto Project.
- Supported Machines
  - The meta-mchp-common layer provides support for various Microchip platforms. For detailed information about supported machines, please refer to the documentation in the relevant sub-layers:
    - MPU layer README
- Prerequisites
  - Before starting, please refer to the "Required Packages for Build Host" section in the Yocto Project documentation to install the required dependencies for the build environment:

Note: Make sure to install git-lfs and repo in addition to the required packages for your Linux distribution.

For instance, on Ubuntu® or Debian®, these packages need to be installed on your development host:

```
sudo apt-get install gawk wget git-core git-lfs diffstat unzip texinfo gcc-multilib \
     build-essential chrpath socat cpio python3 python3-pip python3-pexpect \
     xz-utils debianutils iputils-ping python3-git python3-jinja2 libegl1-mesa libsdl1.2-dev \
     pylint3 xterm repo
```

- Usage
  - To integrate this layer into your Yocto Project build environment:

1. Clone the necessary repositories.

   Create an empty directory to hold the workspace:

```
mkdir yocto-dev
cd yocto-dev
```

   Use the repo tool to fetch all the required repositories.

   Note: Make sure to install the repo utility first.

```
repo init -u https://github.com/linux4microchip/meta-mchp-manifest.git -b <branch> -m <target>/default.xml
```

   Replace and with the Yocto Project release branch and the manifest required. For example:

```
repo init -u https://github.com/linux4microchip/meta-mchp-manifest.git -b scarthgap -m mpu/default.xml
```

   Fetch all the required repositories using the following repo command:

```
repo sync
```

2. Initialize the build environment.

   The meta-mchp repository provides sample configuration templates that help set up BitBake layers and key configuration files in the Yocto Project build directory.
   Set the **TEMPLATECONF** environment variable to point to the appropriate configuration template before initializing the build environment:

```
export TEMPLATECONF=${TEMPLATECONF:-../meta-mchp/<meta-layer>/conf/templates/default}
```

   Replace meta-layer above with the desired layer based on your target platform. For example:

```
export TEMPLATECONF=${TEMPLATECONF:-../meta-mchp/meta-mchp-mpu/meta-mchp-mpu-apps/conf/templates/default}
```

   Note: Setting TEMPLATECONF is only needed the first time you will run the source command.

   Then initialize the Yocto Project build environment:

```
source openembedded-core/oe-init-build-env
```

3. Set the target machine and build the image.

```
MACHINE=<machine> bitbake core-image-minimal
```

   Each sub-layer provides several images that include demos and applications tailored for its respective platform.

   For more information on the supported images, please refer to the MPU layer README.

   - Layer dependencies

     This layer depends on the following layers:

```
- meta-openembedded
  - URI: git://git.openembedded.org/meta-openembedded
  - Layers: meta-oe, meta-networking, meta-python

- openembedded-core
  - URI: git://git.openembedded.org/openembedded-core
  - Layers: meta
```

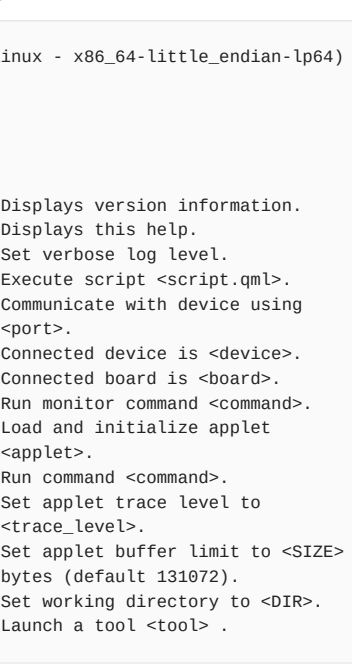
   For information on the specific revisions used, refer to the meta-mchp manifest repository.

- Licensing
  - The contents of this layer are licensed under the MIT License. See **COPYING.MIT** for details.
- Contributing
  - If you want to contribute changes, you can send GitHub pull requests.

Back to Top

Using SAM-BA to flash components to board

NAND Flash Demo - Memory Map



Install SAM-BA Software In Your PC

We maintain information about SAM-BA in the "*SoftwareTools*" page.

Use SAM-BA software v3.9.y onwards. You can download it from the "*SAM-BA v3.9.1*" release page.

Launch SAM-BA tools

- According to the Connect the USB to the board before launching SAM-BA section make sure that the chip can execute the SAM-BA Monitor.

In addition to the Qt® 5 QML language for scripting used for flashing the demos, the most common SAM-BA software actions can be done using the SAM-BA software command line.

For browsing information on the SAM-BA software command line usage, please see the command line documentation available in the SAM-BA software installation directory, doc/index.html or doc/cmdline.html.

SAM-BA software includes a command-line interface that provides support for the most common actions:

- Reading/writing to arbitrary memory addresses and/or peripherals
- Uploading applets and using them to erase/read/write external memories

The command line interface is designed to be self-documenting.

The main commands can be listed using the `--help` command:

```
SAM-BA Command Line Interface Tool v3.9.1 (linux - x86_64-little_endian-lp64)
Copyright 2025 Microchip Technology

Usage: sam-ba [options]

Options:
  -v, --version                          Displays version information.
  -h, --help                             Displays this help.
  -l, --loglevel <log_level[:options:...]> Set verbose log level.
  -x, --execute <script.qml>             Execute script <script.qml>.
  -p, --port <port[:options:...]>        Communicate with device using
                                         <port>.
  -d, --device <device[:options:...]>    Connected device is <device>.
  -b, --board <board[:options:...]>      Connected board is <board>.
  -m, --monitor <command[:options:...]>  Run monitor command <command>.
  -a, --applet <applet[:options:...]>    Load and initialize applet
                                         <applet>.
  -c, --command <command[:args:...]>     Run command <command>.
  -t, --tracelevel <trace_level>         Set applet trace level to
                                         <trace_level>.
  -L, --applet-buffer-limit <SIZE>       Set applet buffer limit to <SIZE>
                                         bytes (default 131072).
  -w, --working-directory <DIR>          Set working directory to <DIR>.
  -u, --utils <tool[:args:...]>          Launch a tool <tool> .
```

If you get the following error, it indicates a symbol mismatch or unresolved symbol when the `sam-ba` executable is being loaded:

```
$ sam-ba -v
sam-ba: symbol lookup error: sam-ba: undefined symbol: _ZdlPvm, version Qt_5
```

Check for Qt libraries inside the extracted SAM-BA software directory.

```
$ cd path/to/sam-ba
$ export LD_LIBRARY_PATH=$(pwd)/lib:$LD_LIBRARY_PATH
```

Additional help can be obtained for most commands by supplying a "help" parameter that will display their usage.

For example, "*sam-ba --port help*" will display:

```
Known ports: j-link, serial, secure
```

Commands that take an argument with options (port, monitor, applet) will display even more documentation when called with "help" as the option value.

For example, "*sam-ba --port serial:help*" will display:

```
Syntax:
    serial:[<port>]:[<baudrate>]
Examples:
    serial                serial port (will use first AT91 USB if found otherwise first serial port)
    serial:COM80          serial port on COM80
    serial:ttyUSB0:57600  serial port on /dev/ttyUSB0, baudrate 57600
```

Configure NAND ECC

Using default PMECC parameters

when choosing the board variant with the -b parameter of SAM-BA, the default PMECC configuration for the NAND populated on the board is valid. You can verify its value by running the command that reads one byte in a dummy file (named test.bin in the following command):

before accessing the NAND flash, you have to initialize the external RAM:

```
# sam-ba -p serial -b sam9xx5-ek -a extram
```

```
# sam-ba -p serial -b sam9xx5-ek -a nandflash -c read:test.bin:0:1
Opening serial port 'ttyACM0'
Connection opened.
Compatible device detected: SAM9G35.
Detected memory size is 268435456 bytes.
Page size is 2048 bytes.
Buffer is 131072 bytes (64 pages) at address 0x2000a000.
NAND header value is 0xc0c00405.
Supported erase block sizes: 128KB
Executing command 'read:test.bin:0:1'
Read 1 bytes at address 0x00000000 (100.00%)
Connection closed.
```

You can figure out that the default PMECC parameter for this sam9xx5-ek board is 0xc0c00405.

Note that if you connect a serial console to the SoC RomCode default UART, you can see even more details about the NAND ECC parameters given by the SAM-BA Applet:

```
Applet 'NAND Flash' from softpack 3.8 (v3.8).
Initializing NAND ioSet1 Bus Width 16
PMECC configuration: 0xc0c00405
Sector size: 512
Sectors per page: 4
Spare size: 64
ECC bits: 2
ECC offset: 48
ECC size: 16
PMECC enabled
Buffer Address: 0x2000a000
Buffer Size: 67065856 bytes
NAND applet initialized successfully.
```

If you want to change the default PMECC parameters you can simply specify another value on the SAM-BA command line with the -a nandflash argument as shown below:

```
# sam-ba -p serial -b sam9xx5-ek -a nandflash:help
Syntax: nandflash:[<ioset>]:[<bus_width>]:[<header>]
Parameters:
    ioset      I/O set
    bus_width  NAND bus width (8/16)
    header     NAND header value
Examples:
    nandflash                 use default board settings
    nandflash:2:8:0xc0098da5  use fully custom settings (IOSET2, 8-bit bus, header is 0xc0098da5)
    nandflash:::0xc0098da5    use default board settings but force header to 0xc0098da5
For information on NAND header values, please refer to SAMA5D4 datasheet section \"12.4.4 Detailed Memory Boot Procedures\".
```

By reading this in-line documentation we can specify the NAND PMECC parameter with this command:

```
# sam-ba -p serial -b sam9xx5-ek -a nandflash:::0xc0c00405
Opening serial port 'ttyACM0'
Connection opened.
Compatible device detected: SAM9G35.
Detected memory size is 268435456 bytes.
Page size is 2048 bytes.
Buffer is 131072 bytes (64 pages) at address 0x2000a000.
NAND header value is 0xc0c00405.
Supported erase block sizes: 128KB
Connection closed.
```

Programming components into NAND

Program AT91Bootstrap binary

Run SAM-BA with USB connection (equivalent to serial) and erase the beginning of the NAND flash **and then** write AT91Bootstrap binary:

```
# sam-ba -p serial -b sam9xx5-ek -a nandflash -c erase::0x40000 -c writeboot:at91bootstrap.bin
Opening serial port 'ttyACM0'
Connection opened.
Compatible device detected: SAM9G35.
Detected memory size is 268435456 bytes.
Page size is 2048 bytes.
Buffer is 131072 bytes (64 pages) at address 0x2000a000.
NAND header value is 0xc0c00405.
Supported erase block sizes: 128KB
Executing command 'erase::0x40000'
Erased 131072 bytes at address 0x00000000 (50.00%)
Erased 131072 bytes at address 0x00020000 (100.00%)
Executing command 'writeboot:at91bootstrap.bin'
Prepended NAND header prefix (0xc0c00405)
Appending 1380 bytes of padding to fill the last written page
Wrote 14336 bytes at address 0x00000000 (100.00%)
Connection closed.
```

Program U-Boot binary

Run SAM-BA with USB connection (equivalent to serial) and erase the U-Boot section in the NAND flash memory map **and then** write U-Boot binary:

```
# sam-ba -p serial -b sam9xx5-ek -a nandflash -c erase:0x40000:0x80000 -c write:u-boot.bin:0x40000
Opening serial port 'ttyACM0'
Connection opened.
Compatible device detected: SAM9G35.
Detected memory size is 268435456 bytes.
Page size is 2048 bytes.
Buffer is 131072 bytes (64 pages) at address 0x2000a000.
NAND header value is 0xc0c00405.
Supported erase block sizes: 128KB
Executing command 'erase:0x40000:0x80000'
Erased 131072 bytes at address 0x00040000 (25.00%)
Erased 131072 bytes at address 0x00060000 (50.00%)
Erased 131072 bytes at address 0x00080000 (75.00%)
Erased 131072 bytes at address 0x000a0000 (100.00%)
Executing command 'write:u-boot.bin:0x40000'
Appending 640 bytes of padding to fill the last written page
Wrote 131072 bytes at address 0x00040000 (24.81%)
Wrote 131072 bytes at address 0x00060000 (49.61%)
Wrote 131072 bytes at address 0x00080000 (74.42%)
Wrote 131072 bytes at address 0x000a0000 (99.22%)
Wrote 4096 bytes at address 0x000c0000 (100.00%)
Connection closed.
```

Recent FAQ

- Compiling Linux Kernel fails looking at OpenSSL header files
- Convert SAM-BA Scripts
- Yocto Project FAQ
- Use of the AT91 ADC driver
- Using ISI (Image Sensor Interface) in Linux4SAM 6.0 and later
- Real Time solutions on AT91SAM SoC
- SD card boot for AT91SAM SoC
- How to manually select (or enforce) the PDA at boot time
- Shutdown/Wake Up FAQ
- U-Boot FAQ
- USB Gadget Configuration
- External Component on External Bus Interface

0 Likes   Tags: _$prv_

Information contained on this site regarding device applications and the like is provided only for your convenience and may be superseded by updates. It is your responsibility to ensure that your application meets with your specifications. MICROCHIP MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND WHETHER EXPRESS OR IMPLIED, WRITTEN OR ORAL, STATUTORY OR OTHERWISE, RELATED TO THE INFORMATION, INCLUDING BUT NOT LIMITED TO ITS CONDITION, QUALITY, PERFORMANCE, MERCHANTABILITY OR FITNESS FOR PURPOSE. Microchip disclaims all liability arising from this information and its use. Use of Microchip devices in life support and/or safety applications is entirely at the buyer's risk, and the buyer agrees to defend, indemnify and hold harmless Microchip from any and all damages, claims, suits, or expenses resulting from such use. No licenses are conveyed, implicitly or otherwise, under any Microchip intellectual property rights.

© 2026 Microchip Technology, Inc.